

NAME : BERELLI PRIYANKA

HALL TICKET NO : 2303A52468

BATCH NO : 42

Task 1: Responsive Homepage Layout for a Startup Website

Prompt : Create a responsive startup homepage in HTML with a header (navigation links), main content session, and footer, using semantic tags and a meta viewport for responsiveness CODE:

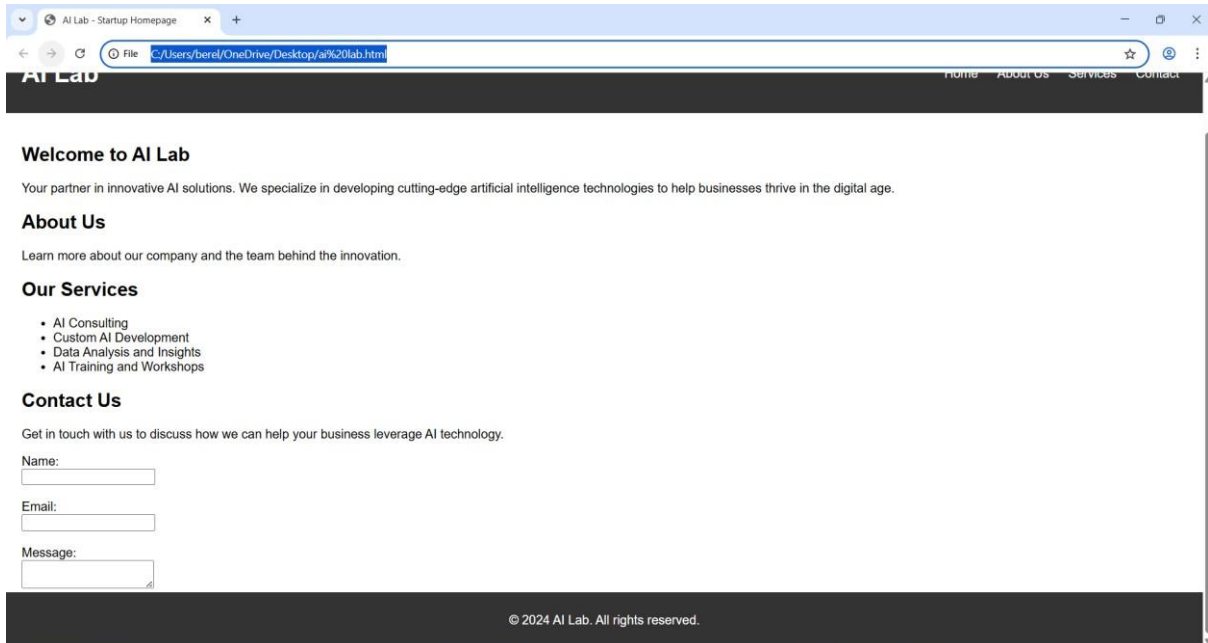
```
create a responsive startup homepage in HTML with a header (navigation links),  
main content session and a footer, using semantic tags and a meta viewport for  
responsiveness  
<!DOCTYPE html><html lang="en">  
<head>    <meta charset="UTF-8">  
        <meta name="viewport" content="width=device-width, initial-scale=1.0">  
        <title>AI Lab - Startup Homepage</title>  
        <style>            body {  
font-family: Arial, sans-serif;  
margin: 0;                padding: 0;  
                }            header {  
background-color: #333;                color:  
white;                padding: 10px 20px;  
display: flex;                justify-content:  
space-between;                align-items:  
center;  
                }            nav a {  
color: white;                text-  
decoration: none;  
margin: 0 10px;  
                }            main {  
padding: 20px;  
                }  
            footer {  
                background-color: #333;  
color: white;                text-  
align: center;                padding:  
10px 0;                position: fixed;
```

```

        width: 100%;
bottom: 0;
    }
    @media (max-width: 600px) {
header {
            flex-direction:
column;
            align-items:
flex-start;
        }
        nav {
margin-top: 10px;
        }
    }
}
</style></head><body>
<header>
    <h1>AI Lab</h1>
    <nav>
        <a href="#home">Home</a>
        <a href="#about">About Us</a>
        <a href="#services">Services</a>
        <a href="#contact">Contact</a>
    </nav>
    </header>
    <main>
        <section id="home">
            <h2>Welcome to AI Lab</h2>
            <p>Your partner in innovative AI solutions. We specialize in
developing cutting-edge artificial intelligence technologies to help
businesses thrive in the digital age.</p>
        </section>
        <section
id="about">
            <h2>About Us</h2>
            <p>Learn more about our company and the team behind the
innovation.</p>
        </section>
        <section id="services">
            <h2>Our Services</h2>
            <ul>
                <li>AI Consulting</li>
                <li>Custom AI Development</li>
                <li>Data Analysis and Insights</li>
                <li>AI Training and Workshops</li>
            </ul>
        </section>
        <section id="contact">
            <h2>Contact Us</h2>
            <p>Get in touch with us to discuss
how we can help your business leverage AI technology.</p>
            <form>
                <label for="name">Name:</label><br>
                <input type="text" id="name" name="name"><br><br>
                <label for="email">Email:</label><br>
                <input type="email" id="email" name="email"><br><br>
                <label for="message">Message:</label><br>
                <textarea id="message" name="message"></textarea><br><br>
                <input type="submit" value="Submit">
            </form>
        </section>
    </main>
    <footer>
        <p>
>&copy; 2024 AI Lab. All rights reserved.</p>
    </footer>
</body></html>

```

OUTPUT :



LINK : <file:///C:/Users/berel/OneDrive/Desktop/ai%20lab.html>

EXPLANATION :

This task is about creating a responsive startup homepage using HTML and CSS. The webpage should include a header with navigation links, a main content section, and a footer with basic information. CSS media queries are used to make the layout adapt to different screen sizes like desktop, tablet, and mobile so the website looks clean and organized on all devices.

Task 2: Interactive Call-to-Action Button

Prompt : Create a web page with an HTML call-to-action button and JavaScript that shows an alert message when the button is clicked, with clear comments explaining the JS code and no page reload.

CODE:

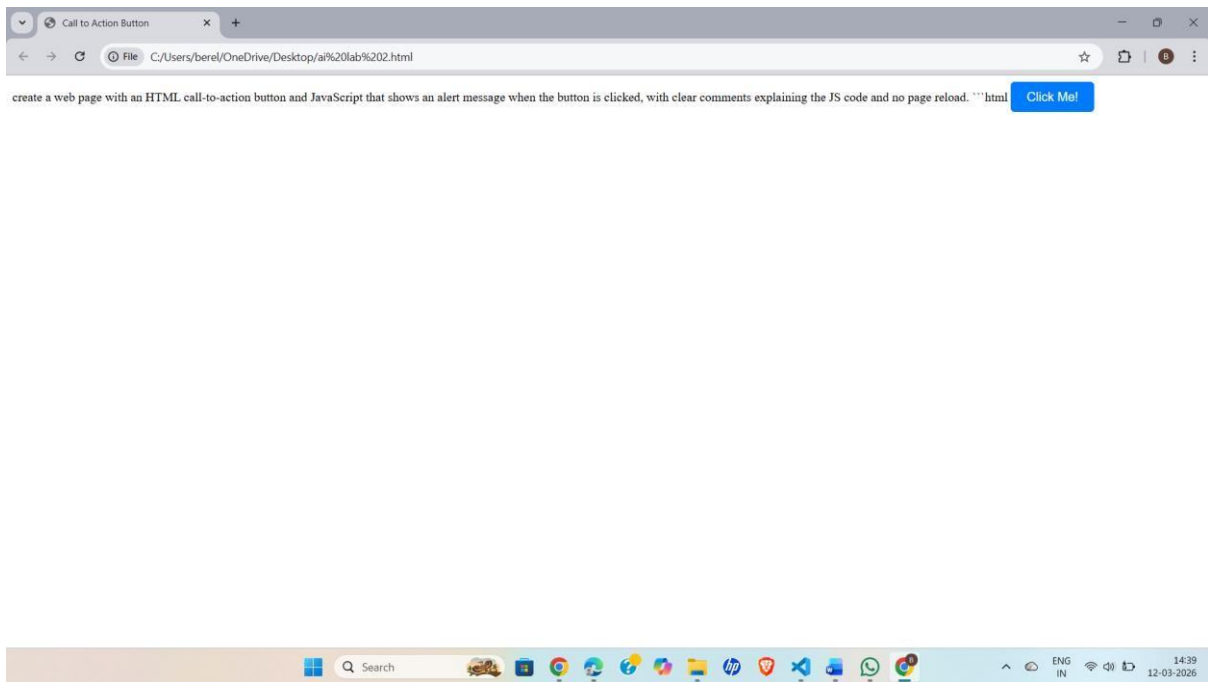
```
create a web page with an HTML call-to-action button and JavaScript that shows an alert message when the button is clicked, with clear comments explaining the JS code and no page reload.
```html<!DOCTYPE html>
<html lang="en">
<head>
```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Call to Action Button</title>
<style>
 /* Basic styling for the button */
 .cta-button {
padding: 10px 20px; font-
size: 16px; color: white;
background-color: #007BFF;
border: none; border-
radius: 5px; cursor:
pointer;
 }
 .cta-button:hover { background-color:
#0056b3; /* Darker blue on hover */
 } </style>
</head>
<body>
 <!-- Call-to-action button -->
 <button class="cta-button" id="ctaButton">Click Me!</button>
 <script>
 // Get the button element by its ID const
ctaButton = document.getElementById('ctaButton');
 // Add an event listener to the button that listens for 'click'
events ctaButton.addEventListener('click', function() {
 // When the button is clicked, show an alert message
alert('Button was clicked!'); // This will display a pop-up alert with the
message });
 </script>
</body>
</html>

```

OUTPUT:



LINK : <file:///C:/Users/berel/OneDrive/Desktop/ai%20lab%202.html>

#### EXPLANATION :

This task involves adding a call-to-action button to a webpage using HTML and using JavaScript to make it interactive. When the user clicks the button, a message or alert should appear without reloading the page. The JavaScript code should also include comments to explain how the button click works.

### Task 3: Contact Form with Client-Side Validation

Prompt: Create a simple HTML contact form for a company website with clientside validation using JavaScript.

**CODE :** Create a simple HTML contact form for a company website with clientside validation using JavaScript.

```
```html<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>Contact Form</title>
<style>
    /* Basic styling for the form */
    .contact-form {
width: 400px;          margin: 0 auto;
padding: 20px;        border: 1px solid
#ccc;                border-radius: 5px;
    }
    .form-group {
bottom: 15px;        margin-
    }
    .form-group label {
display: block;      margin-bottom:
5px;
    }
    .form-group input, .form-group textarea {
width: 100%;         padding: 8px;
    box-sizing: border-box;
    }
    .submit-button {
padding:
10px 20px;          font-size: 16px;
color: white;        background-color:
#28a745;            border: none;
border-radius: 5px;  cursor:
pointer;
    }
    .submit-button:hover {
        background-color: #218838; /* Darker green on hover */
    }
</style>
</head>
<body>
    <!-- Contact form -->
    <form class="contact-form" id="contactForm">
        <div class="form-group"
            <label for="name">Name:</label>
            <input type="text" id="name" name="name" required>
        </div>
        <div class="form-group">
            <label for="email">Email:</label>
            <input type="email" id="email" name="email" required>
        </div>
        <div class="form-group">

```

```

        <label for="message">Message:</label>
        <textarea id="message" name="message" rows="4"
required></textarea>
    </div>
    <button type="submit" class="submit-button">Submit</button>
</form>
<script>
    // Get the form element by its ID      const
contactForm = document.getElementById('contactForm');
    // Add an event listener to the form that listens for 'submit' events
contactForm.addEventListener('submit', function(event) {
    // Prevent the default form submission behavior (page
reload)      event.preventDefault();
    // Get the values from the form fields      const name =
document.getElementById('name').value.trim();      const email =
document.getElementById('email').value.trim();      const message =
document.getElementById('message').value.trim();
    // Simple client-side validation      if (name === '' ||
email === '' || message === '') {      alert('Please fill in all
fields.');
```

// Alert if any field is empty

return; // Exit

the function if validation fails }

```

    // If validation passes, show a success message
alert('Thank you for contacting us, ' + name + '! We will get back to you
shortly.');
```

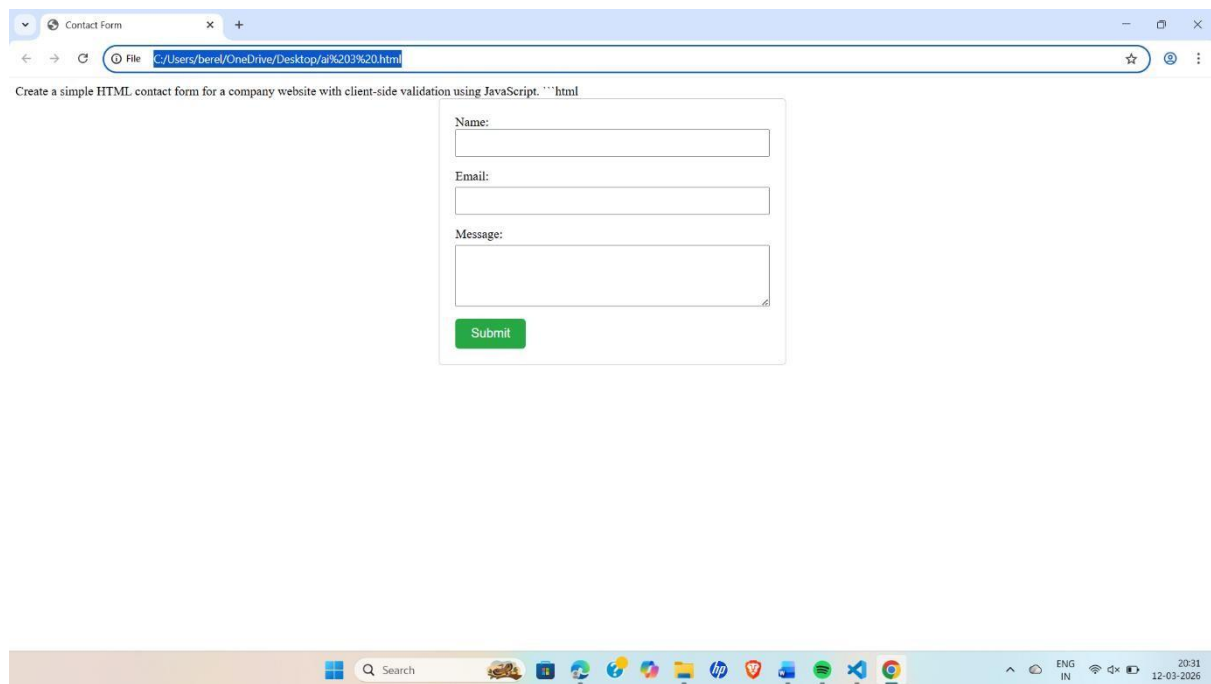
// Optionally, you can reset the form after successful submission

```

contactForm.reset();
    });
</script>
</body>
</html>

```

OUTPUT :



The screenshot shows a web browser window titled "Contact Form". The address bar displays the file path: `File C:/Users/berel/OneDrive/Desktop/ai%203%20.html`. Below the address bar, a subtitle reads: "Create a simple HTML contact form for a company website with client-side validation using JavaScript. ''html". The contact form itself is centered and contains three input fields: "Name:", "Email:", and "Message:". Each field is a simple text box. Below these fields is a green "Submit" button. The browser's taskbar at the bottom shows various application icons and the system clock indicating 20:31 on 12-03-2026.

LINK : <file:///C:/Users/berel/OneDrive/Desktop/ai%203%20.html>

EXPLANATION : This task involves creating a contact form for a company website that allows users to send queries. The form includes three fields: Name, Email, and Message.

Client-side validation using JavaScript is used to check the user input before the form is submitted. The script ensures that:

- All fields are not empty.
- The email format is valid.

If the user enters incorrect or empty information, inline error messages are displayed near the fields. If all inputs are correct, a success message is shown without reloading the page.

This improves data accuracy, user experience, and prevents invalid submissions.

Task 4: Dynamic List Management for a Product Page

Prompt: Create a simple dynamic product list webpage using HTML, CSS, and JavaScript.

CODE :

```
# Create a simple dynamic product list webpage using HTML, CSS, and
JavaScript.
```html<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Product List</title>
 <style>
 /* Basic styling for the product list */
 .product-list {
max-width: 600px;
margin: 20px auto;
padding: 20px; border:
1px solid #ccc;
border-radius: 5px;
 }
 .product-item {
display: flex; justify-content:
space-between; padding: 10px 0;
border-bottom: 1px solid #eee;
 }
 .product-item:last-child {
 border-bottom: none; /* Remove border from the last item */
 }
 .add-product-form {
display: flex; justify-content:
space-between; margin-top:
20px;
 }
 .add-product-form input
{
 padding: 8px;
width: 45%;
 box-sizing: border-box;
 }
 .add-product-form button {
padding: 8px 16px; background-
color: #007BFF; color: white;
 }
 </style>
</head>
<body>
 <div class="product-list">
 <div class="product-item">
 <div>Product 1</div>
 <div>Add to Cart</div>
 </div>
 <div class="product-item">
 <div>Product 2</div>
 <div>Add to Cart</div>
 </div>
 <div class="product-item">
 <div>Product 3</div>
 <div>Add to Cart</div>
 </div>
 </div>
 <div class="add-product-form">
 <input type="text" value="Add New Product" />
 <button>Add Product</button>
 </div>
</body>
</html>
```



```

 border: none;
border-radius: 5px; cursor:
pointer;
 }
 .add-product-form button:hover { background-
color: #0056b3; /* Darker blue on hover */ }
 </style>
</head>
<body>
 <div class="product-list" id="productList">
 <h2>Product List</h2>
 <!-- Product items will be added here dynamically -->
 </div>

 <form class="add-product-form" id="addProductForm">
 <input type="text" id="productName" placeholder="Product Name"
required>
 <input type="number" id="productPrice" placeholder="Product Price"
required>
 <button type="submit">Add Product</button>
 </form>

 <script>
 // Get references to the product list and form elements
const productList = document.getElementById('productList'); const
addProductForm = document.getElementById('addProductForm'); const
productNameInput = document.getElementById('productName'); const
productPriceInput = document.getElementById('productPrice');
 // Function to add a new product to the list
function addProduct(name, price) {
 // Create a new div element for the product item
const productItem = document.createElement('div');
productItem.classList.add('product-item');
 // Set the inner HTML of the product item with the name and price
productItem.innerHTML = `
 ${name}
 ${price.toFixed(2)}
 `;

 // Append the new product item to the product list
productList.appendChild(productItem);
}

 // Event listener for form submission to add a new product
addProductForm.addEventListener('submit', function(event) {

```

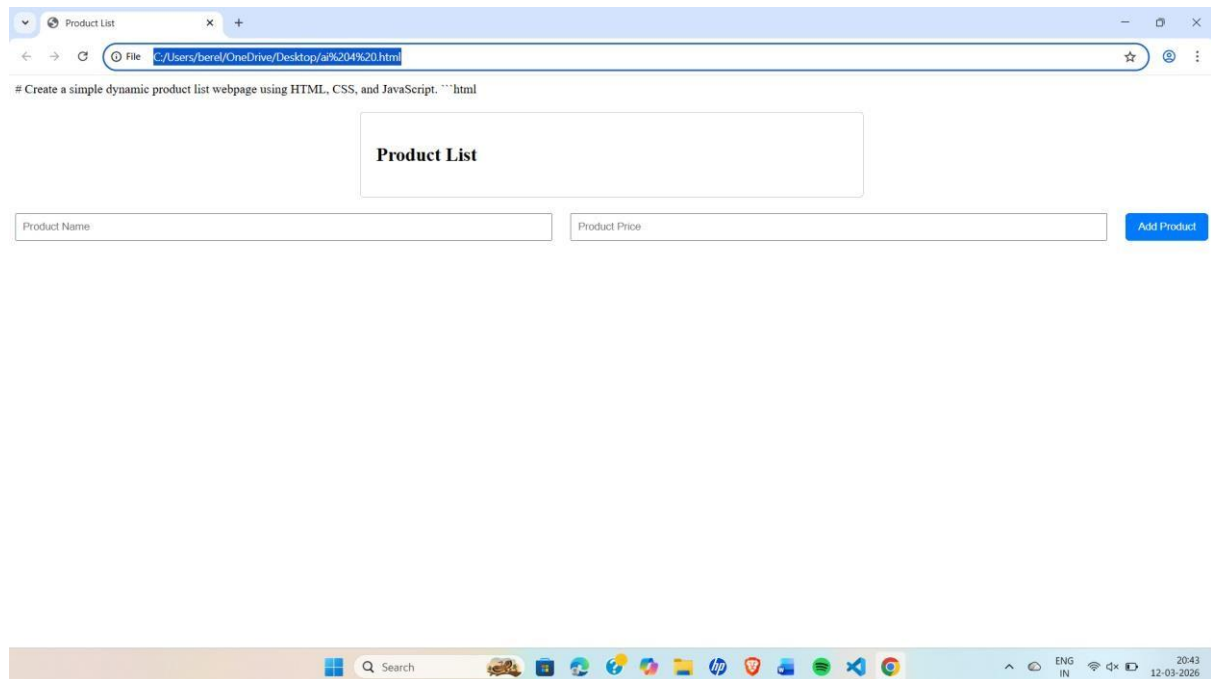
```

 event.preventDefault(); // Prevent the default form submission
behavior

 // Get the values from the input fields
const name = productNameInput.value.trim(); const
price = parseFloat(productPriceInput.value);
 // Validate the input values if (name &&
!isNaN(price) && price > 0) { addProduct(name, price); //
Add the new product to the list
 // Clear the input fields after adding the product
productNameInput.value = ''; productPriceInput.value
= '';
 } else { alert('Please enter a valid product name
and price.');// Show an alert if validation fails
 }
 });
</script>
</body>
</html>

```

## OUTPUT :



LINK : <file:///C:/Users/berel/OneDrive/Desktop/ai%204%20.html>

EXPLANATION : This task creates a dynamic product list using HTML and JavaScript. Users can add new items to the list using the Add button and remove the last item using the Remove button. JavaScript updates the DOM dynamically, so the list changes instantly without reloading the page. Task 5: Modal Popup for User Notifications

Prompt : Create a modal popup notification using HTML, CSS, and JavaScript.

CODE:

```
Create a modal popup notification using HTML, CSS, and JavaScript.
```html<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Modal Popup Notification</title>
  <style>
    /* Basic styling for the modal */
    .modal {
      display: none; /* Hidden by default */
      position: fixed; /* Stay in place */
      z-index: 1; /* Sit
on top */
      left: 0;
      top: 0;
      width: 100%; /* Full width */
      height: 100%; /* Full
height */
      overflow: auto; /* Enable scroll if needed */
      background-color: rgb(0,0,0); /* Fallback color */
      background-color: rgba(0,0,0,0.4); /* Black w/ opacity */
    }
    .modal-content {
      background-color: #fefefe;
      margin: 15%
auto; /* 15% from the top and centered */
      padding:
20px;
      border: 1px solid #888;
      width: 80%; /* Could be more or less, depending on screen size */
    }
    .close {
      color: #aaa;
      float: right;
      font-size: 28px;
      font-weight: bold;
    }
    .close:hover,
.close:focus {
      color: black;

```

```

        text-decoration:          none;
cursor: pointer;
    }
</style>
</head>
<body>
    <!-- Trigger/Open The Modal -->
    <button id="myBtn">Open Modal</button>
    <!-- The Modal -->
    <div id="myModal" class="modal">
        <!-- Modal content -->
        <div class="modal-content">
            <span class="close">&times;</span>
            <p>This is a modal popup notification!</p>
        </div>
    </div>

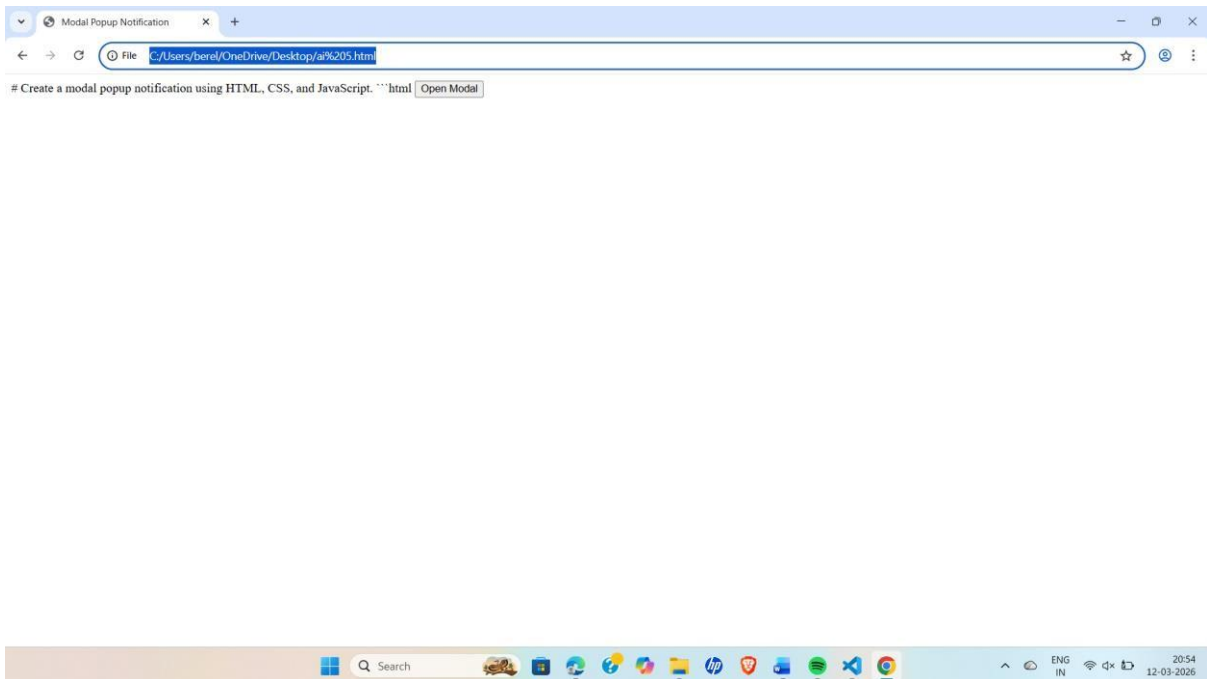
    <script>
        // Get the modal element          const modal =
document.getElementById('myModal');
        // Get the button that opens the modal
const btn = document.getElementById('myBtn');          //
Get the <span> element that closes the modal
        const span = document.getElementsByClassName('close')[0];
        // When the user clicks the button, open the modal
btn.onclick = function() {          modal.style.display =
'block'; // Show the modal          }

        // When the user clicks on <span> (x), close the modal
span.onclick = function() {          modal.style.display =
'none'; // Hide the modal          }

        // When the user clicks anywhere outside of the modal, close it
window.onclick = function(event) {          if (event.target == modal) {
modal.style.display = 'none'; // Hide the modal if clicked outside of it
}          }
    </script>
</body>
</html>

```

OUTPUT :



LINK : <file:///C:/Users/berel/OneDrive/Desktop/ai%205.html> EXPLANATION

:

This task creates a modal popup notification for a website using HTML, CSS, and JavaScript. The modal appears when the user clicks the “Open Modal” button and displays a message on top of the page with a semi-transparent background overlay. The popup can be closed using a close button or by clicking outside the modal, making the notification interactive and userfriendly.